

Lab 6. Merkle Trees for Distributed Audit Logs

Contents

1	Audit Logs in Distributed Systems	1
1.1	Challenges	2
2	Merkle Trees: a solution for integrity assurance	2
2.1	Example	3
3	Implementation Tasks	5
3.1	Merkle Hash Tree	5
3.2	Log Server	6
3.3	Auditor	7
3.4	Distributed deployment	7
4	Deliverables and evaluation	8

1 Audit Logs in Distributed Systems

Audit logs are used to record security events and maintain a shared, append-only history of events across distributed parties. Examples include the Bitcoin blockchain and Certificate Transparency global certificate log.

In distributed systems, databases are often spread across multiple servers. This improves availability but introduces a challenge: how can users trust the data they retrieve? For example, in a security log system, if a hacker tampers with log entries, how can we detect this?

1.1 Challenges

The main challenges in ensuring the integrity of audit logs in distributed systems include:

- **Tampering Resistance:**
 - Distributed systems involve multiple nodes that can log data. A malicious entity may tamper with the logs to hide unauthorized actions or fraud.
 - Ensuring that logs remain unaltered is crucial for trust and accountability.
- **Scalability:**
 - Logs in distributed systems can grow very large. Verifying the integrity of an entire log file or dataset can become computationally expensive.
 - Efficient methods to verify subsets of logs without reprocessing the entire log are necessary.
- **Decentralization:**
 - Logs are generated and stored across multiple nodes, which increases the difficulty of maintaining a single source of truth.
 - Synchronization and consensus among nodes about the state of logs are challenging.
- **Secure Verification:**
 - A mechanism is needed to verify the integrity of logs without exposing sensitive data contained in them.
 - Verification processes should be efficient to allow frequent checks.
- **Auditor Trust:**
 - External auditors may need to verify logs without fully trusting the entities providing the logs.
 - Proving the logs' authenticity and integrity in a trustless or semi-trust environment is critical.

2 Merkle Trees: a solution for integrity assurance

Merkle trees are a cryptographic data structure that can effectively address these challenges. They provide a way to verify the integrity of data without having to examine every individual piece of data.

Merkle trees provide two primary benefits:

- **Audit Paths:** Enable efficient verification of individual records with $O(\log n)$ hashes.
- **Consistency Proofs:** Allow verification of log extensions without examining the entire dataset.

How Merkle Trees Work:

1. Hashing: Each log entry is hashed to produce a fixed-size hash value.
2. Tree Construction:
 - Hash values are paired and hashed together, creating a new hash.
 - This process continues until a single root hash is obtained.
3. Integrity Verification:
 - To verify the integrity of a specific log entry, its hash and the root hash are compared.
 - If the root hash is unchanged, it indicates that the log entry has not been tampered with.

Uses of Merkle-Tree-Based Audit Logging Systems:

- *Data Integrity*: **Google's Certificate Transparency** logs ensure all SSL/TLS certificates are publicly recorded, enabling users to detect unauthorized certificates.
- *Blockchain Applications*: **Bitcoin** and **Ethereum** use Merkle trees to validate transactions.
- *Tamper Detection*: HashiCorp's **Vault** uses Merkle trees to detect unauthorized changes in secure logs.
- *Distributed Systems*: Amazon **DynamoDB** uses Merkle trees for data replication consistency.
- *Transparency and Accountability*: **Let's Encrypt** logs certificates using Merkle trees to provide proof of issuance.
- *IoT*: Lightweight blockchains like **IoTeX** use Merkle trees for IoT device data integrity.
- *Software Distribution*: Google's **Android Verified Boot** ensures the integrity of operating system images using Merkle trees.

2.1 Example

A Merkle hash tree is a data structure used to efficiently and securely verify the integrity of a dataset. Consider a log containing 8 events: $e_1, e_2, \dots, e_8$. Figure 1 illustrates the Merkle tree for this log.

1. **Leaf Nodes**: The bottom level of the tree consists of hashes of the individual events $e_1, e_2, \dots, e_8$.
2. **Intermediate Nodes**: Moving up the tree, pairs of neighboring hashes are incrementally combined and hashed to form the parent nodes. For example, the hash of e_1 and e_2 forms $h_{1..2}$, while the hash of e_3 and e_4 forms $h_{3..4}$.
3. **Root Node**: The final root hash, $h_{1..8}$, represents the hash of the entire dataset. It is derived from the hashes of its left and right subtrees, $h_{1..4}$ and $h_{5..8}$.

Efficiency of Merkle Trees

While constructing a Merkle tree for n records requires approximately $2n - 1$ hash computations—one for each leaf and internal node—this overhead is manageable. In contrast, a simple

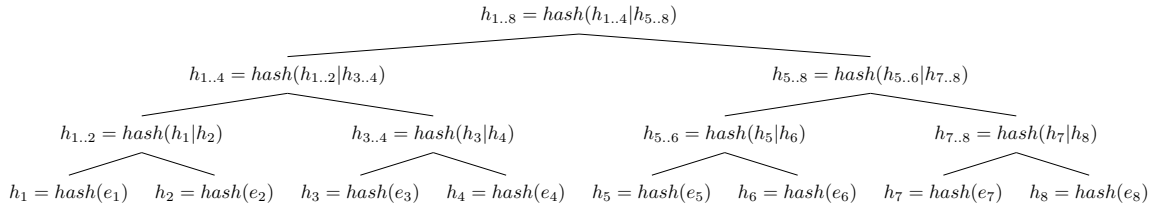


Figure 1: Merkle tree for the log of 8 events

sequential hash of the database requires n computations. However, the true advantage of a Merkle tree lies in its ability to efficiently verify individual records or subsets of the dataset, requiring only $O(\log n)$ hashes for validation.

Audit Paths The audit path allows verification of a specific record in the log without needing the entire tree.

1. **Integrity Verification:** Suppose a user retrieves event e_4 from an untrusted source but has the root hash $h_{1..8}$ from a trusted server. To verify the integrity of e_4 , the untrusted source must provide:
 - the sibling hash h_3 ,
 - the parent hash $h_{1..2}$, and
 - the hash $h_{5..8}$ from the other subtree.

These values form the **audit path**, which enables the user to reconstruct the path from e_4 to $h_{1..8}$ as shown in Figure 2.

2. **Efficiency:** Verifying a single record requires only $O(\log n)$ hashes to be transmitted and computed. This is far more efficient than transmitting the entire tree or dataset.

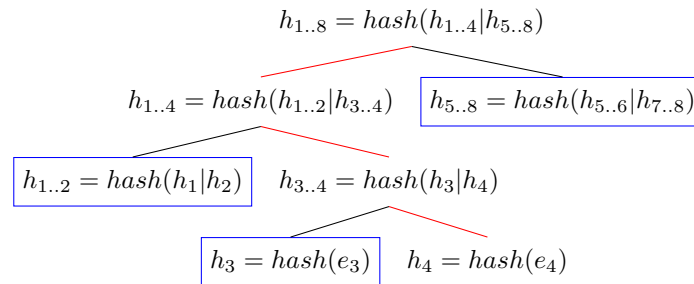


Figure 2: Merkle tree - the audit path

Consistency Proofs Logs often grow over time as new events are added. Since Merkle trees are append-only structures, they allow efficient verification that a new log is an extension of an old one.

1. **Scenario:** Suppose a user has the root hash of a log containing $n = 6$ events (Figure 3) and wants to verify that a new log with $m = 8$ events extends the original without tampering.
2. **Proof Requirements:** The user must be provided with the hashes $h_{1..4}$, $h_{5..6}$, and $h_{7..8}$.
3. **Verification Process:**
 - Using $h_{1..4}$ and $h_{5..6}$, the user reconstructs $h_{1..6}$, the root hash of the original log.
 - Using $h_{1..4}$, $h_{5..6}$, and $h_{7..8}$, the user reconstructs $h_{1..8}$, the root hash of the new log.

By confirming that $h_{1..8}$ correctly extends $h_{1..6}$, the user ensures that the log has grown without modifications to the original records.

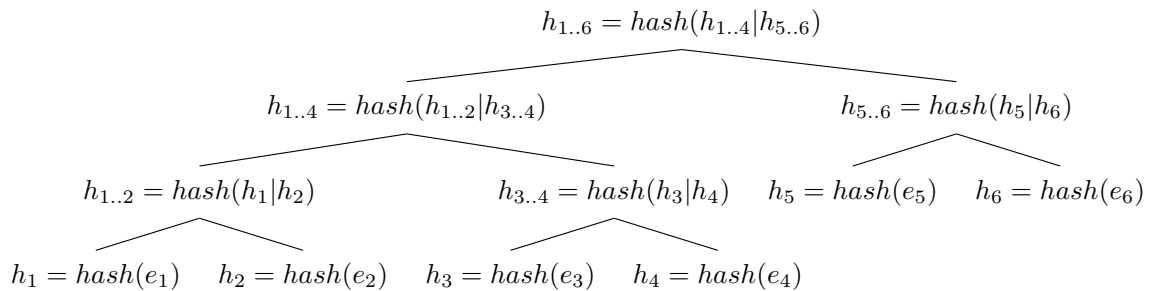


Figure 3: Merkle tree for the log of 6 events

3 Implementation Tasks

The goal is to build a system using Merkle Trees to ensure audit logs are tamper-proof. This will include:

1. A MerkleTree class to represent the tree.
2. A LogServer to manage logs using the tree.
3. An Auditor to verify log integrity.

We consider logs to be simple **text files**, where each line corresponds to an event.

3.1 Merkle Hash Tree

A Merkle Tree summarizes log events as a single root hash. Each node represents the hash of its content:

- **Leaves:** Hashes of individual log events

The constructor for a leaf takes a string containing a single event of the log, converts it to bytes, prepends it with the single byte 0x00 and computes its hash.

$$h_i = \text{SHA-256}(0x00 \mid \text{utf8}(e_i))$$

- **Internal nodes:** Combine two child hashes

The constructor for an internal node takes two Merkle trees for the left and right subtrees. It concatenates the hashes of its two subtrees, prepends them with 0x01, and hashes the result.

$$h_{1..8} = \text{SHA-256}(0x01 \mid h_{1..4} \mid h_{5..8})$$

Using different byte-tags in the hashes helps prevent some hash collisions.

Implementation:

1. Write a `MerkleTree` class:

- Store a node's hash, its left and right children, and the range of log entries it represents (e.g., $h_{5..8}$ covers records 5 to 8).
- Compute a leaf's hash as `SHA256(0x00 | utf-8(event))`.
- Compute an internal node's hash as `SHA256(0x01 | left_hash | right_hash)`.

2. Implement a method to build the full Merkle Tree from a list of log events.

To compute the event hash, the `SHA256` function from `java.security.MessageDigest` can be used:

```
MessageDigest digest = MessageDigest.getInstance("SHA256");
byte [] hash = digest.digest(leaf);
```

To convert a string to a byte array, you can use `text.getBytes("UTF-8")`.

3.2 Log Server

The **Log Server** manages the Merkle Tree and allows users to:

1. Retrieve the root hash: The hash summarizing all log events.
2. Append events: Add one or more events to the log and update the tree.
3. Generate an audit path: Provide the minimal set of hashes needed to verify an event's integrity.
4. Generate a consistency proof: Prove that a new log extends an old log without modification.

Implementation:

- Build the Merkle Tree from a file (each line is a log event).
- Define a method that returns the current root hash.
- Define methods to append events to the log one-at-a-time, or as a batch of events to be appended all at once.
- Implement a method `genPath` that given the current Merkle tree of size n and any index $i \leq n$ returns the audit path for the i -th event, required to verify the hashes of e_i . For example, in the Merkle tree of 1..8 events, `genPath(4)` for e_4 would return the array of hashes $[h_3, h_{1..2}, h_{5..8}]$ (Fig. 2).
- Implement a method `genProof` that given the current Merkle tree of size n and any previous tree size $m \leq n$ returns the consistency proof that the current tree extends the previous tree. For example, in the Merkle tree of 1..8 events, `genProof(6)` would return the array of hashes $[h_{7..8}, h_{5..6}, h_{1..4}]$.

3.3 Auditor

The **Auditor** ensures logs are tamper-proof without trusting the server. It verifies:

- Membership: Check if a given event exists in the log.
- Consistency: Check if a new log extends an old log without modification.

Implementation:

- Retrieve the log's size and root hash during initialization.
- Implement a method `isMember` that verifies that an event exists in the log using its audit path (the path should begin with a hash of the event and end with the root hash, and all hashes on the path should be correctly computed).
- Test your code by verifying membership for various events.

3.4 Distributed deployment

For a basic distributed deployment of the Merkle-tree based audit logging system (with a single log server), the goal is to set up a system where the **Log Server** is accessible by multiple clients for appending logs and querying audit paths.

Architecture overview:

- Log Server: A centralized server hosting logs and maintaining the Merkle Tree.
 - Accepts log appends from clients.

- Provides root hash, audit paths, and consistency proofs.
- Clients: Entities that append events or query the Log Server for audit paths/consistency proofs.
- Communication: gRPC, RPC or RMIs for interaction between clients and the server.

4 Deliverables and evaluation

Report with output screenshots and **explanations** for each part Source code

The project will be evaluated on the basis of (a) the clean design and implementation, (b) the careful analysis of the complexity of various methods, (c) and on the quality of tests that have been used to verify the solution.